TP MI-GUIDE CONSTRUCTION PROGRESSIVE D'UN SYSTEME DE BOISSONS

Nom : Aya Lmasoudi

Lien du dépôt GitHub : https://github.com/lmasoudiaya-11/Exs_python/tree/main/Tp_2

1. Description du problème :

Dans ce projet, l'objectif est de créer un petit système en Python permettant de gérer des commandes de boissons dans un café.
Le programme doit permettre de créer différents types de boissons (comme le café ou le thé), d'ajouter des ingrédients supplémentaires comme le lait, le sucre ou le caramel, puis de les ajouter à une commande associée à un client.

Le système doit également permettre de calculer le prix total d'une commande, d'afficher le contenu de la commande et de gérer un programme de fidélité permettant d'ajouter des points au client après la validation de la commande.

L'idée principale est d'utiliser les concepts de programmation orientée objet (POO) en Python pour structurer le programme de manière claire et modulaire.

2. Architecture du programme :

Pour organiser le programme, plusieurs classes ont été créées afin de représenter les différents éléments du système.

Classe Boisson

La classe Boisson est une classe abstraite qui représente une boisson générale.
Elle définit deux méthodes abstraites :

- cout() : retourne le prix de la boisson
- description() : retourne une description de la boisson

Toutes les boissons doivent hériter de cette classe.

```
############# Partie 1 : Création d'une structure de base

from abc import ABC , abstractmethod

class Boisson (ABC) :

  @abstractmethod
  def cout(self) :
    pass
  @abstractmethod
  def description(self) :
    pass

  def __add__(self , other) :
    return Combiner_Boisson(self , other)
```

- Classes Cafe et The

Les classes Cafe et The héritent de la classe Boisson.

Elles représentent les boissons de base et définissent :

- leur prix
- leur description

Par exemple :

- Café simple : 2.0 €
- Thé : 1.5 €

```
############# Partie 2 : Création de boissons concrètes

class Cafe(Boisson) :

  def cout(self) :
    return 2.0

  def description(self) :
    return "Café simple"

class The(Boisson) :

  def cout(self) :
    return 1.5

  def description(self) :
    return "Thé"
```

Décorateurs d'ingrédients

Pour ajouter des ingrédients supplémentaires, une classe DecorateurBoisson est utilisée.

Les classes suivantes héritent de cette classe :

- Lait
- Sucre
- Caramel

Ces classes permettent de modifier la boisson originale en ajoutant un ingrédient et en augmentant son prix.

Par exemple :

```
boisson2 = Lait(cafe)
boisson2 = Sucre(boisson2)
```

donne :

Café simple au lait sucré

Cette approche correspond au pattern Decorator.

```
############# Partie 3 : Ajout d'ingrédients

class DecorateurBoisson(Boisson) :

  def __init__(self , boisson) :
    self._boisson = boisson

### Ajout de lait
class Lait(DecorateurBoisson) :

  def cout(self) :
    return self._boisson.cout() + 0.5

  def description(self) :
    return self._boisson.description() + " au lait"
```

```python
### Ajout de sucre
class Sucre(DecorateurBoisson) :

  def cout(self) :
    return self._boisson.cout() + 0.2

  def description(self) :
    return self._boisson.description() + " sucré"

### Ajout de caramel
class Caramel(DecorateurBoisson) :

  def cout(self) :
    return self._boisson.cout() + 0.3

  def description(self) :
    return self._boisson.description() + " au caramel"
```

- Classe Client

La classe Client représente un client du café.

Elle contient :

- nom
- numero
- points_fidelite

Les points de fidélité sont initialisés à 0.

```python
############# Partie 5 : Représentation d'un client

from dataclasses import dataclass

@dataclass
class Client :
  nom : str
  numero : int
  points_fidelite : int = 0
```

Classe Commande

La classe Commande représente une commande passée par un client.

Elle contient :

- le client
- une liste de boissons commandées

Les principales méthodes sont :

- ajouter_boisson() : ajouter une boisson à la commande
- prix_total() : calculer le prix total
- afficher_commande() : afficher les détails de la commande

```
class Commande :
  def __init__(self , client) :
    self._client = client
    self._boissons = []

  def ajouter_boisson(self , boisson) :
     self._boissons.append(boisson)

  def prix_total(self) :
    total = 0
    for boisson in self._boissons :
      total = total + boisson.cout()
    return total

  def afficher_commande(self) :
    print("Client :", self._client.nom)
    print("Boissons commandées :")
    for boisson in self._boissons :
      print( boisson.description(), ":" , boisson.cout(),"€")

    print("Total : ", self.prix_total(), "€")
```

Héritage des commandes

Deux types de commandes ont été créés :

- CommandeSurPlace
- CommandeEmporter

Ces classes héritent de Commande et redéfinissent la méthode afficher_commande() pour indiquer le type de commande

```
class CommandeSurPlace(Commande):
  def afficher_commande(self) :
    print("Commande sur place ")
    super().afficher_commande()

class CommandeEmporter(Commande):
  def afficher_commande(self) :
    print("Commande a emporter ")
    super().afficher_commande()
```

Programme de fidélité

Une classe Fidelite a été créée pour gérer les points de fidélité.

La méthode principale est :

ajouter_points_fidelite()

Elle ajoute des points au client en fonction du montant total de la commande.

```
class Fidelite :
  def ajouter_points_fidelite(self , client , montant) :
    points_fidelite = int(montant)
    client.points_fidelite += points_fidelite
```

Classe CommandeFidele

La classe CommandeFidele utilise l'héritage multiple :

CommandeFidele(Commande, Fidelite)

Elle permet de valider une commande et d'ajouter automatiquement les points de fidélité au client.

```
class CommandeFidele(Commande , Fidelite) :
  def valider_commande(self) :
    total = self.prix_total()
    self.ajouter_points_fidelite(self._client , total )
```

3. Fonctionnalités principales du programme :

Le programme permet de réaliser plusieurs actions :

Création de clients

Il est possible de créer plusieurs clients avec leur nom et leur numéro.

Création de boissons

Le programme permet de créer :

- des cafés
- des thés
- des boissons personnalisées avec des ingrédients

Gestion des commandes

Chaque client peut avoir une commande contenant plusieurs boissons.

Le programme peut :

- ajouter des boissons
- afficher le contenu de la commande
- calculer le prix total

Programme de fidélité

Lorsque la commande est validée, le client reçoit des points de fidélité en fonction du montant de la commande.

4. Exemple d'exécution du programme :

Voici un exemple de fonctionnement du programme.

Création des clients

```
client1 = Client("aya" , 1)
client2 = Client("Narjes" , 2)
```

Création des boissons

```
boisson1 = cafe
boisson2 = Lait(cafe)
boisson2 = Sucre(boisson2)
boisson3 = Caramel(cafe)
boisson4 = the
boisson5 = Lait(the)
boisson5 = Sucre(boisson5)
```

Création d'une commande

```
commande1 = CommandeFidele(client1)
commande2 = Commande(client2)
```

Ajout des boissons

```
commande1.ajouter_boisson(boisson1)
commande1.ajouter_boisson(boisson2)
commande1.ajouter_boisson(boisson3)
commande1.ajouter_boisson(boisson4)
commande1.ajouter_boisson(boisson5)

commande2.ajouter_boisson(boisson3)
commande2.ajouter_boisson(boisson5)
```

Affichage de la commande

Exemple de résultat :

```
Client : aya
Boissons commandées :
Café simple : 2.0 €
Café simple au lait sucré : 2.7 €
Café simple au caramel : 2.3 €
Thé : 1.5 €
Thé au lait sucré : 2.2 €
Total :  10.7 €
Points de fidelite du client  : 10
Client : Narjes
Boissons commandées :
Café simple au caramel : 2.3 €
Thé au lait sucré : 2.2 €
Total :  4.5 €
Points de fidelite du client  : 0
```